



Intelligent Consumer
Technologies | AA 2025/2026

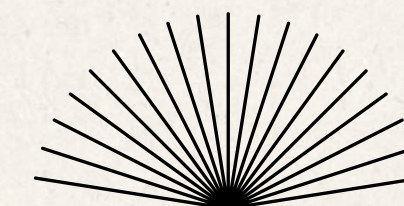
Università degli Studi di Milano-Bicocca

SMART PEEPHOLE

Face Recognition System & Performance Analysis

NAME OF PROJECT:
Smart Peephole

PRESENTED BY:
Chiara Curgu
Lorenzo Mazzone



Summary

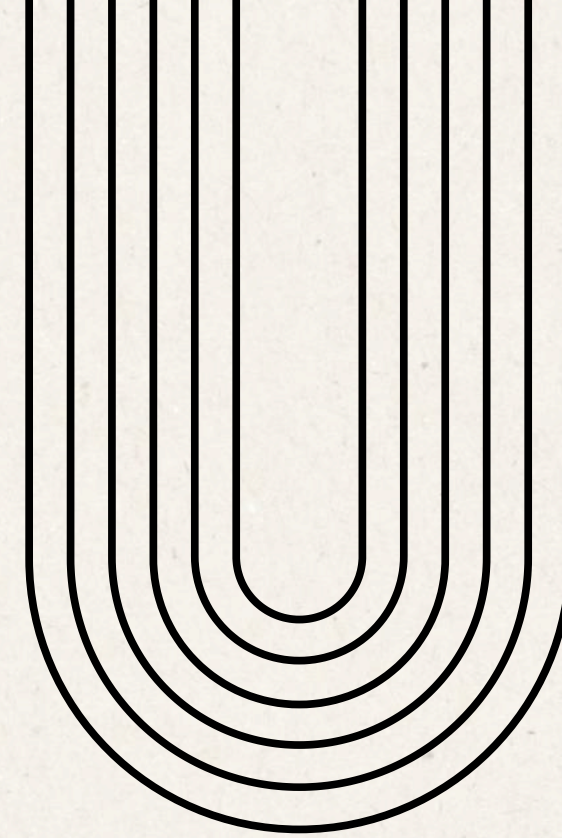
04	Problem Definition
05	Dataset Characteristics
06	System Architecture
07	Identification Logic
08	End-to-end Performances
09	Embedding-only Performances
10	Data Calibration
11	Exhaustive Cross-Validation
12	Streghts and Limitations

Summary

13	Web deployment and user interaction
14	Secure remote access with ngrok
15	Client-server interaction
16	Recognition and enrollment logic
17	Real-world interaction: unknown user alert
18	Strength and limitations - web side
19	Conclusions

Introduction

In the domain of Image Super-Resolution (SR) and Image Restoration, the goal of this project was to reconstruct a high-resolution (HR) image starting from a heavily degraded low-resolution (LR) image (noise, compression artifacts).



Denoise

5 enrolled users with
4-5 images each



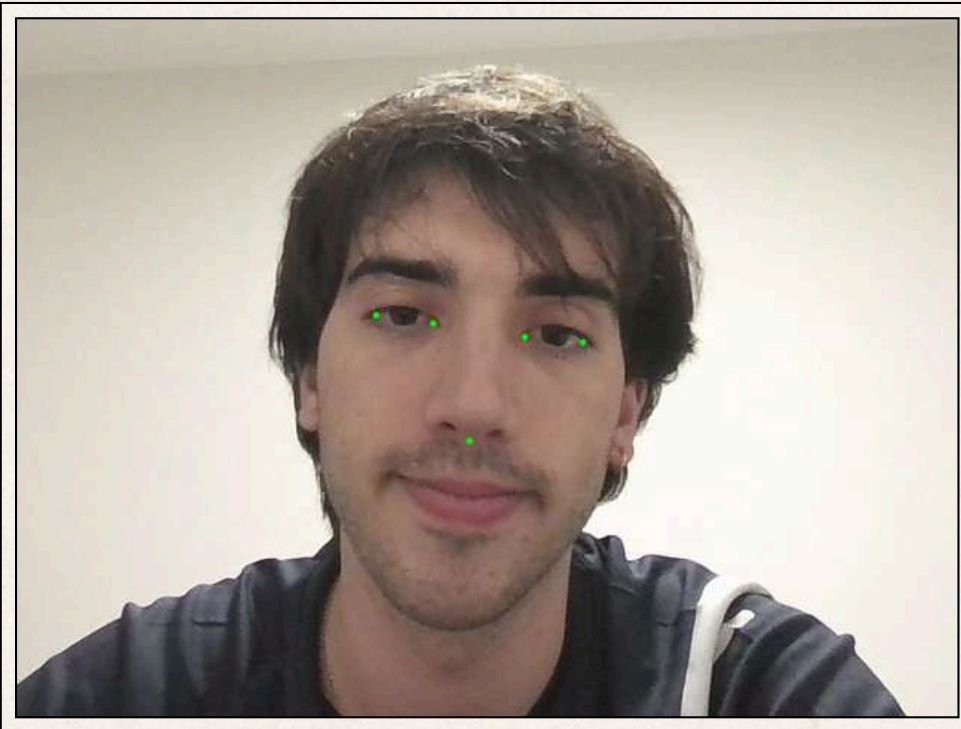
Enhance

Six Additional images of
unknown individuals for testing

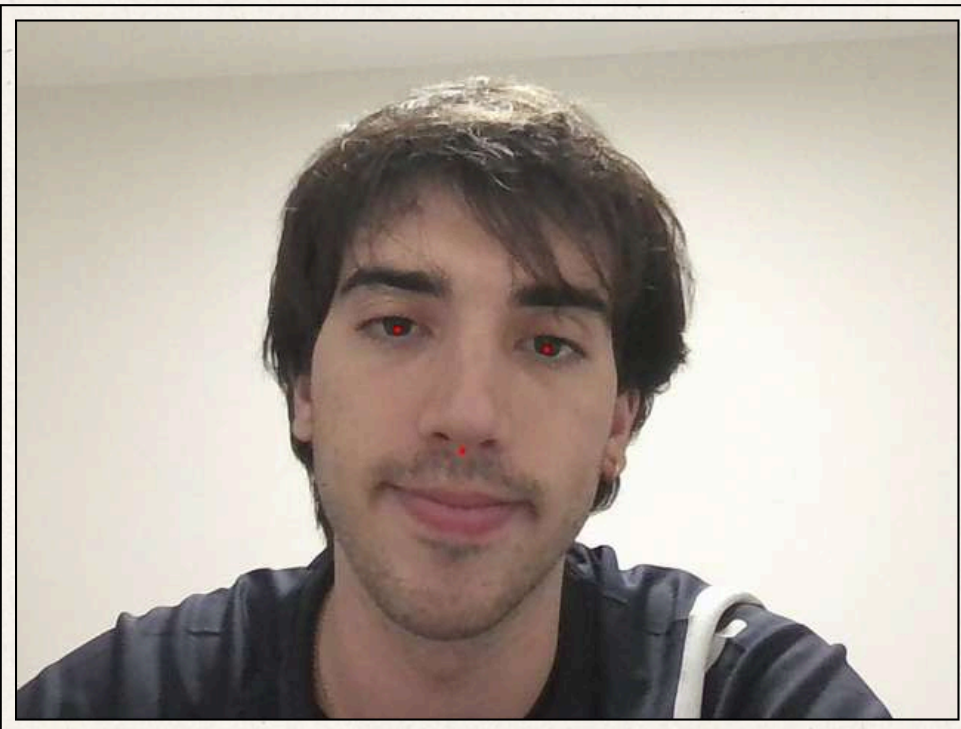


Upscale

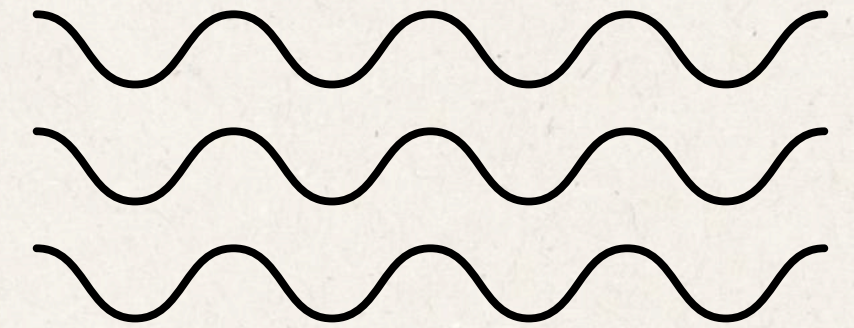
Variability in lighting conditions,
pose, hairstyle, and facial
expression



- 01** Cardinality:
800 images for training, 100 for validation
- 02** High-resolution RGB images, containing diverse scenarios rich in high-frequency textures.



- 03** RAM Management:
difficult to load the entire dataset into memory.
- 05** Implemented a Custom PyTorch DataLoader for dynamic loading (lazy loading) and Tiling.

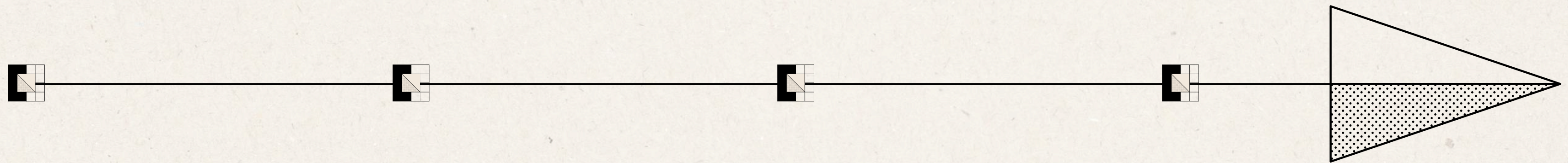


The DIV2K dataset

DIVERse 2K resolution high quality images

Preprocessing and Data Augmentation

How do we choose the correct user?



1

**Preprocessing:
Tensor normalization
in the [0, 1] range:**

Dynamic extraction
of 256x256 patches to
allow batch training
without running out
of VRAM

2

**Geometric Data
Augmentation:**

HorizontalFlip and
VerticalFlip to
increase spatial
variance

3

**Degradation
Pipeline:**

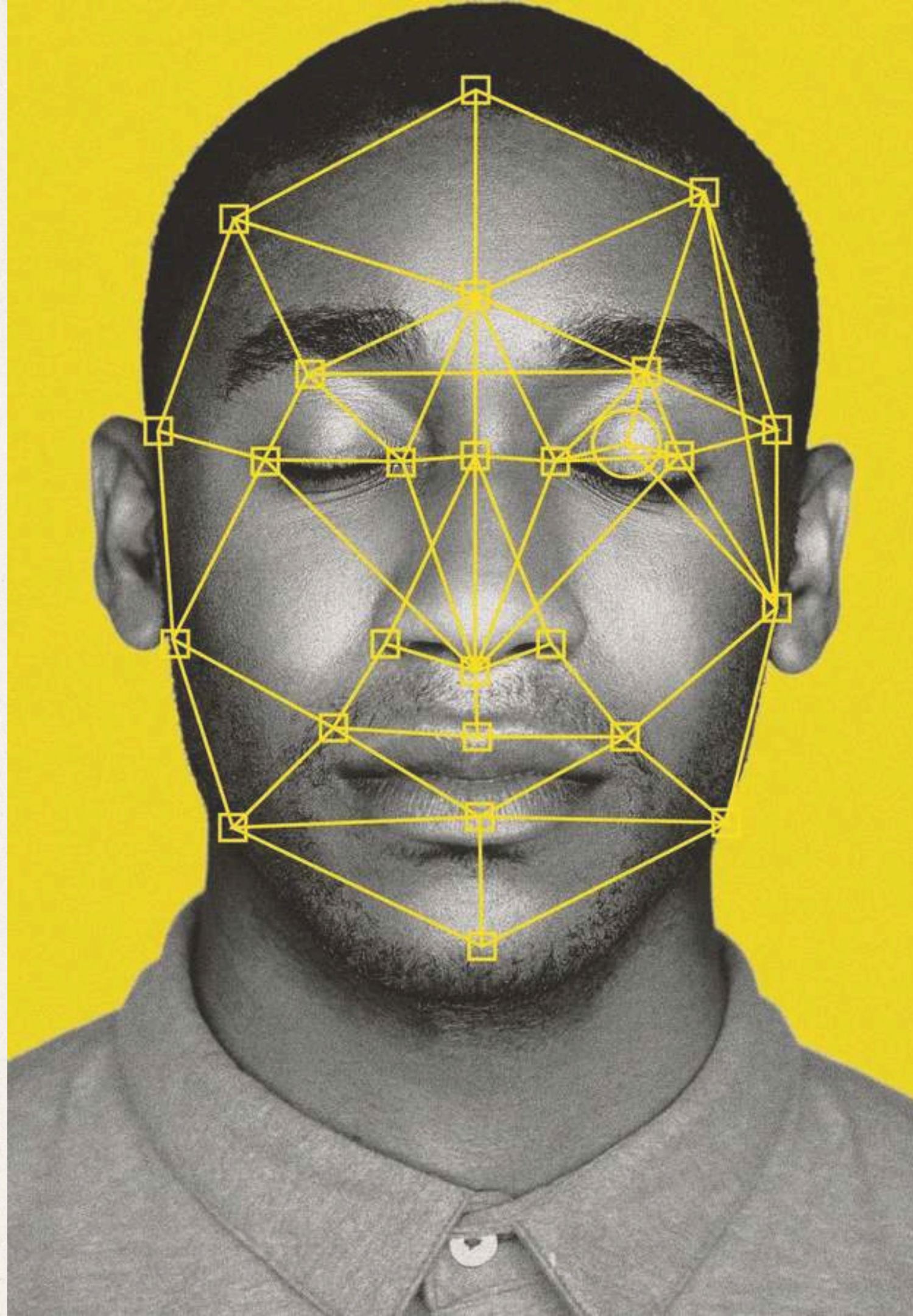
Use of the
albumentations
library to simulate
real-world decay

4

Used techniques:

Downscaling (using
INTER_AREA interpolation to
avoid aliasing),
Addition of GaussNoise,
ImageCompression

- 01 Attempt 1:** Baseline SRGAN. Mixed results. The network struggled to decide whether to remove noise or enhance details.
- 02 Attempt 2:** Self-Attention Integration (ViT inspired). I integrated a Self-Attention module into the generator to provide global context (inspired by Vision Transformers), but the loss conflict persisted.
- 03 Final Solution: Decoupled Restoration Pipeline:** I separated the problem into two cascaded models:
The Denoiser: A network trained strictly to clean the LR image.
The Upscaler (SRGAN + Attention): A network trained strictly for upscaling and texture injection on clean images.



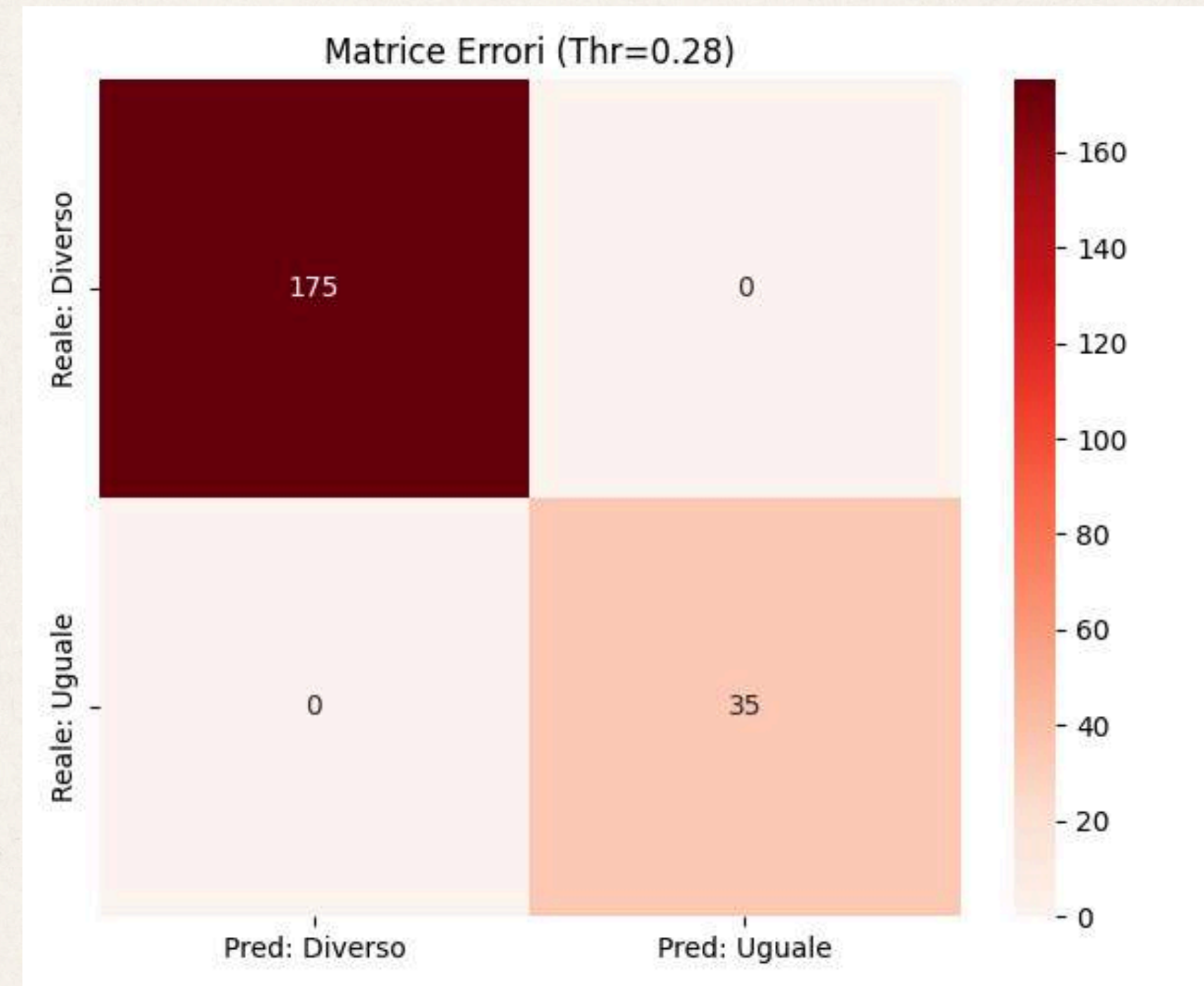
**Proposed
solution**

Architecture, Losses, and Metrics

- Networks: Generator (based on Residual Blocks, Attention, and PixelShuffle) + Discriminator (PatchGAN Classifier)
 - Transfer Learning (The Critic): Use of a pre-trained ResNet50 on ImageNet (frozen weights) to extract deep feature maps.
 - The Hybrid Cost Function (Loss):
 - LossPixel (MSE) = Keeps colors stable.
 - LossPerceptual (ResNet) = Forces the network to mimic complex edges and "style".
 - LossAdversarial (BCE) = Fools the discriminator to inject realism.
 - Evaluation Metrics: PSNR (Peak Signal-to-Noise Ratio) and SSIM (Structural Similarity).
-

Exhaustive cross-validation

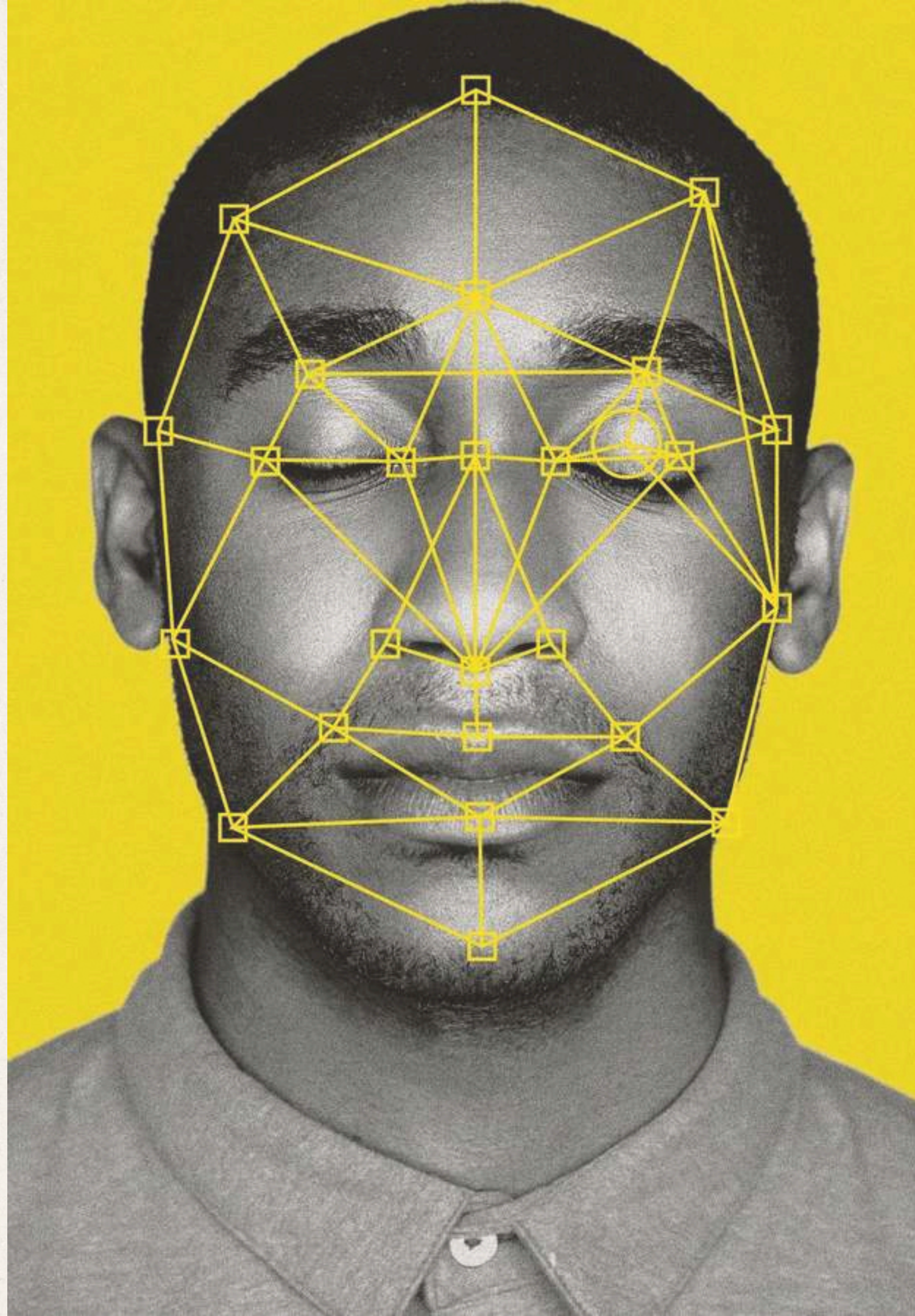
- **Problem 1:** Oversmoothing. The dominant use of MSE produced a "watercolor effect".
- **Problem 2:** Checkerboard Artifacts & Collapse. In stage 2, the GAN produced exploded gradients (colorful checkerboards).
- **The Cause (Domain Gap):** The GAN was trained on mathematically perfect synthetic blur. In inference, it received the neural smoothing of Stage 1. The mathematical mismatch caused the network to hallucinate artifacts.



01 Loss Ablation: Decreased the weight of MSE in favor of Adversarial Loss to unlock texture generation.

02 Domain Gap Resolution (Teacher Forcing):

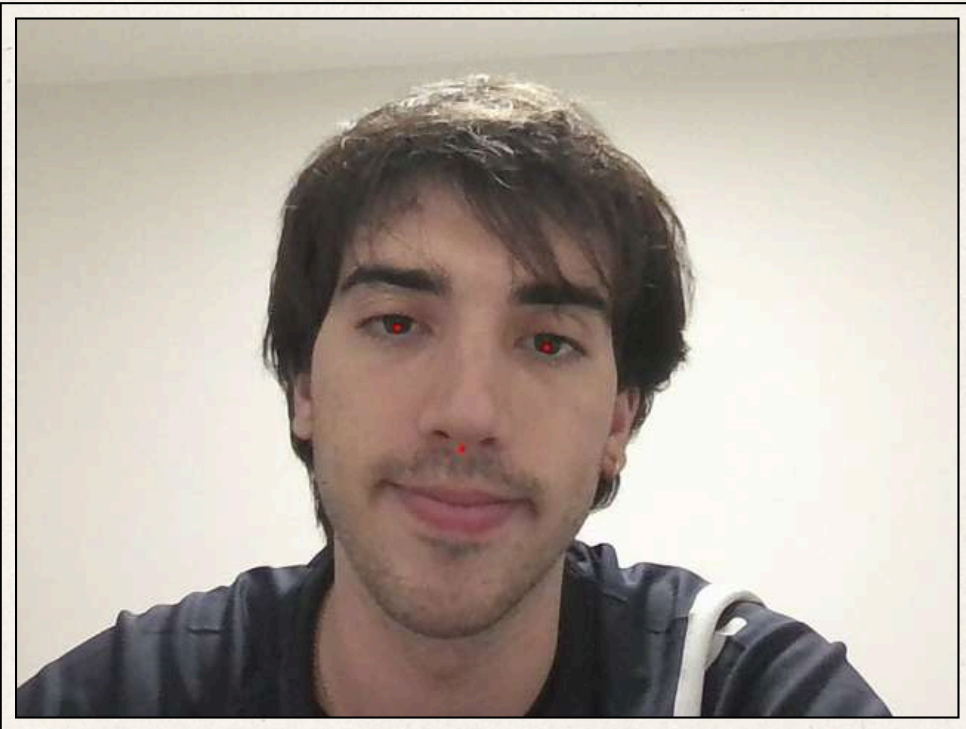
- Developed an offline script with Tiling to process the 800 high-resolution images through Network 1.
- The output of Network 1 became the new training input for Network 2.
- Coordinated geometric cropping at the pixel level.



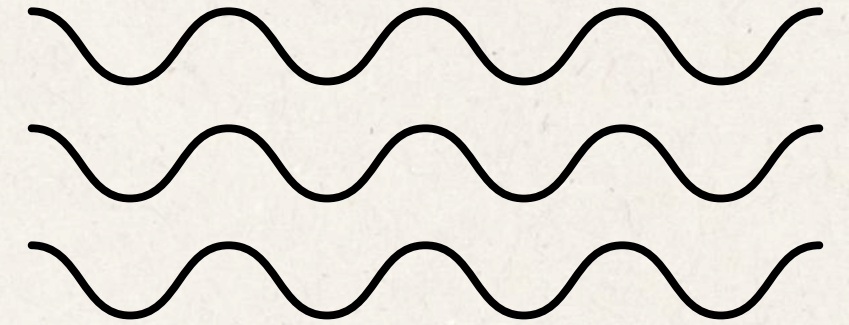
Ablation Study and Domain Alignment



- 01** Simple Baseline:
Bicubic Upscaling
(Results in blurry and noisy images).
- 02** My Model: Custom
Decoupled Pipeline
(Good cleaning, sharp
edges, some limits on
micro-textures).



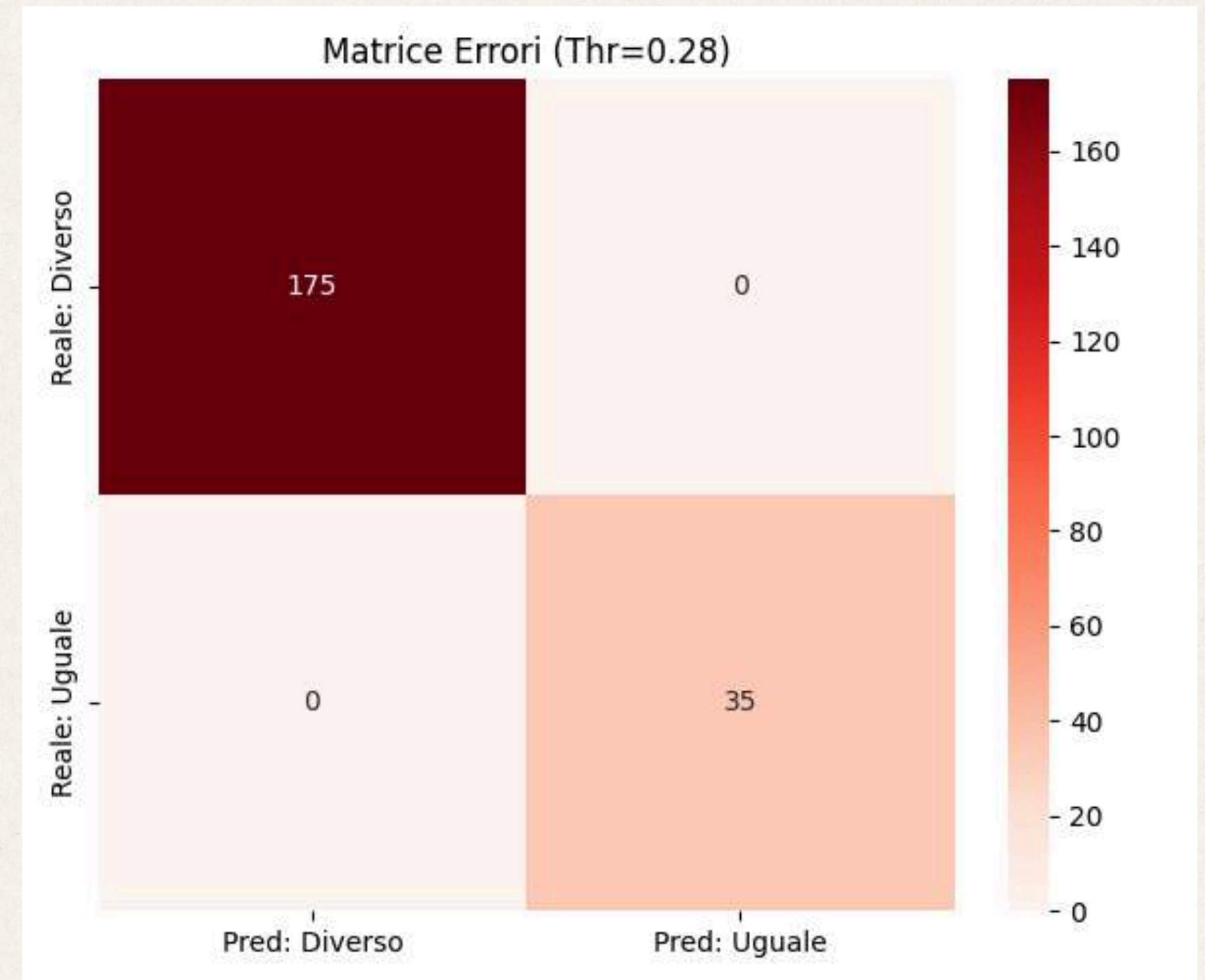
- 03** SOTA 1: SwinIR (Pre-
trained Vision
Transformer).
- 04** SOTA 2: Stable
Diffusion x4 Upscaler
(Pre-trained Latent
Generative Model).



Results and Comparison (Baselines & SOTA)

Interactive Demo and UX

- **Framework:** Streamlit UI.
- **Visual Explainability:** Implemented a dynamic 3-column view to expose the black box: (1) Dirty Original Image → (2) Denoiser Intermediate Output → (3) Final 4K Output.
- **Video Processing:** Experimental support for .mp4 upscaling with decomposition, two-stage frame-by-frame inference, and hardware-accelerated reassembly.
- **SOTA Integration:** Allows the user to select and compare SOTA models in real-time directly from the interface.



Benchmark 1

Evaluation on two CPUs (Intel Core Ultra 7 155H, AMD Ryzen 5 9600x) and one GPU (Nvidia RTX 5070)

Type of analysis

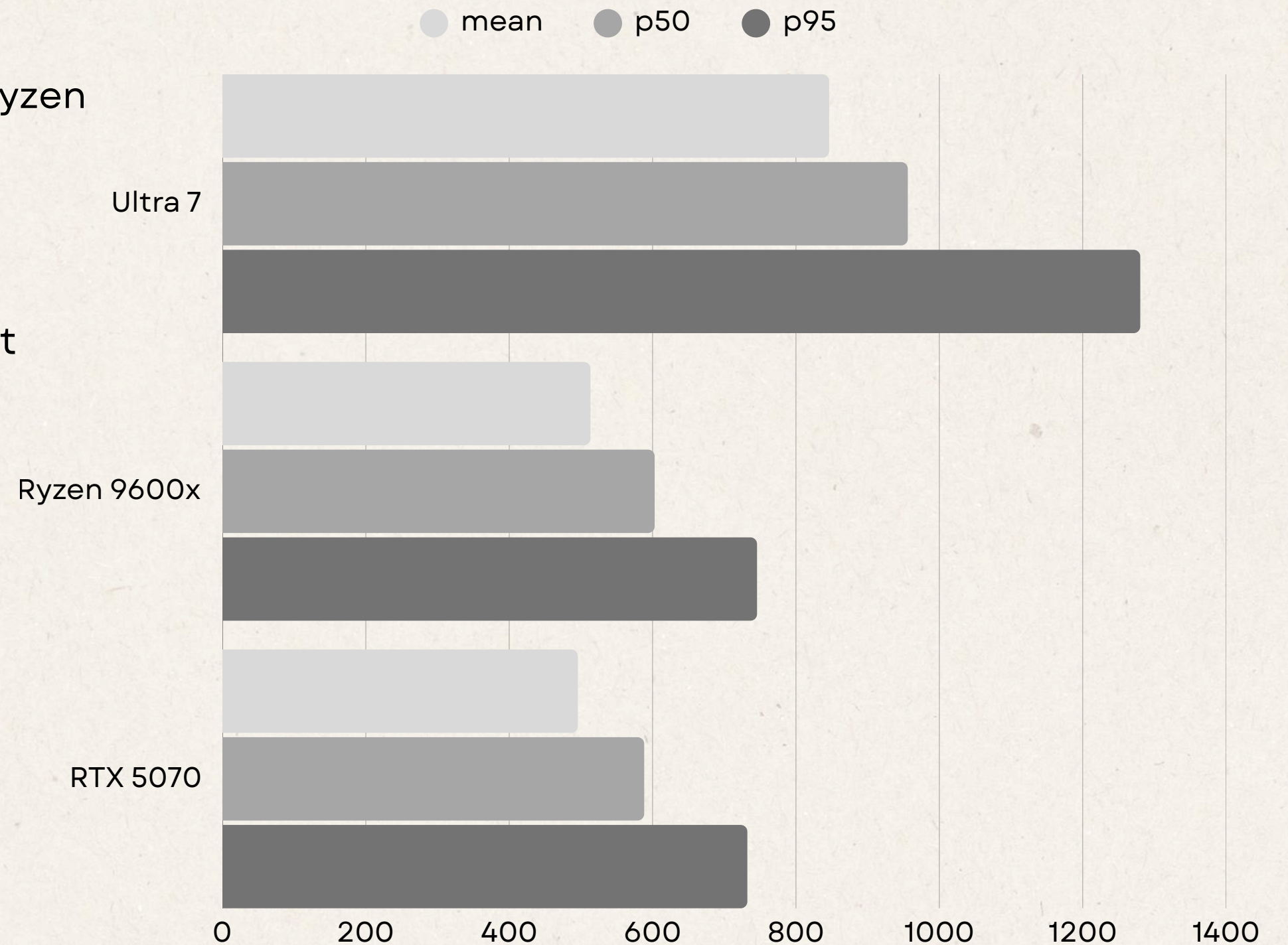
Full pipeline latency measured from raw image input to final identification output (detection, alignment, embedding, and verification included).

CPU performance

CPU performance is dominated by face detection and alignment, resulting in higher overall latency compared to GPU acceleration.

GPU performance

GPU acceleration significantly reduces embedding time, but end-to-end gains remain moderate due to CPU-bound preprocessing stages (Dlib).



Benchmark 2

Evaluation on two CPUs (Intel Core Ultra 7 155H, AMD Ryzen 5 9600x) and one GPU (Nvidia RTX 5070)

Type of analysis

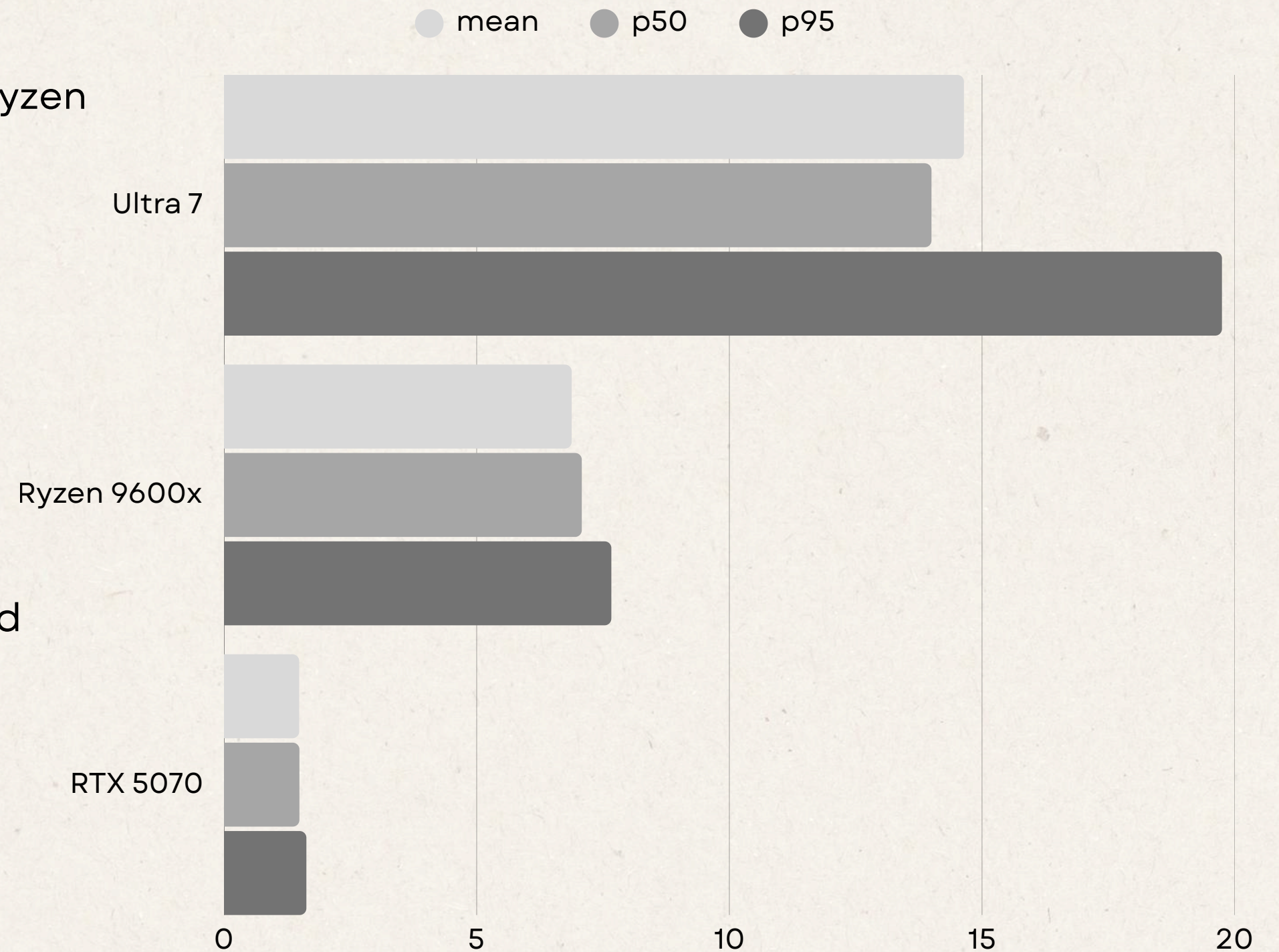
Evaluation focused exclusively on the SphereFace forward pass, isolating neural network inference from preprocessing overhead.

CPU performance

GPU (RTX 5070) achieves a ~4–5× speedup compared to CPU, reducing mean latency to ~1–2 ms per embedding.

GPU performance

Low p50–p95 dispersion indicates stable inference time, suggesting suitability for real-time consumer deployment.



Strenghts and limitations - model side



Results

Stable and discriminative
facial embeddings



Dataset

Limited dataset
size



Performance

Fully functional even on
CPU-only machines



Scalability

Suitable for single-image
authentication, not
high-frame-rate surveillance



Web deployment and user interaction

- 01** Lightweight web application built on top of a Flask backend
- 02** Flask acts as a minimal orchestration layer, exposing HTTP endpoints for face identification and controlled user enrollment
- 03** No additional software is required on the client side: any modern browser is sufficient

Smart Peephole Demo

Live camera

Press **Start camera**. The system auto-identifies after **2 seconds**. Then you can use **Re-identify** manually.

Start cameraRe-identifyStopJoin Telegram

Recognized: **NO**

Person: -

Enroll resident

Type a name, capture at least **4** photos, then press **Enroll**.

CaptureEnrollReset

Captured: 0 / 4

Registered users

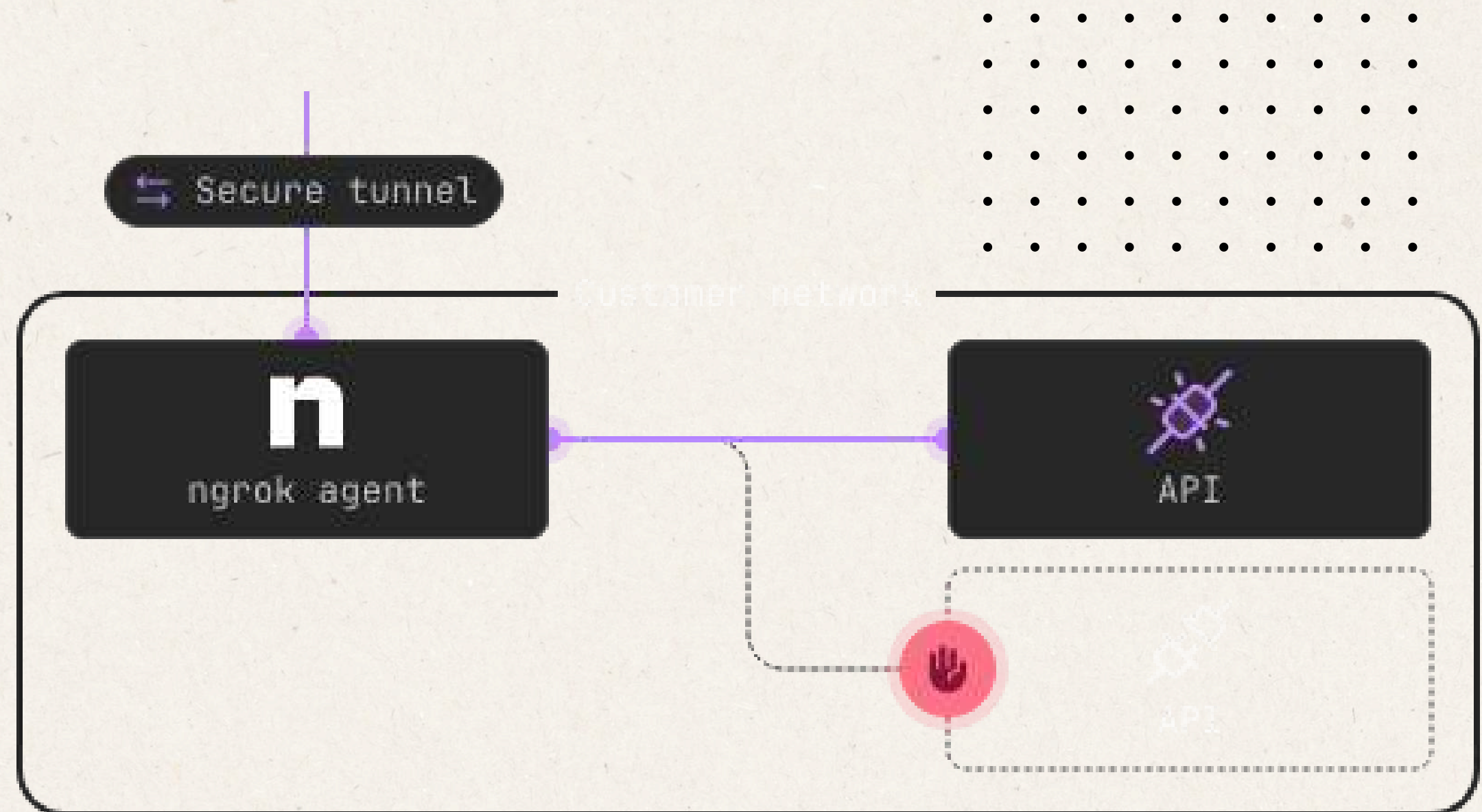
Users currently stored in the local database.

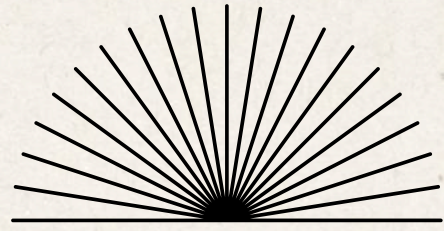
Refresh list

Secure remote access with ngrok

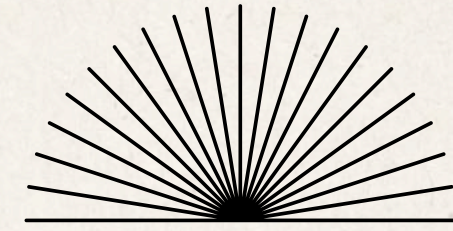
- The system runs entirely on a local machine
- Flask exposes REST APIs for identification and enrollment
- ngrok creates a secure HTTPS tunnel to the local server
- No cloud deployment and no external storage of biometric data

EXPOSURE OF THE LOCAL SYSTEM TO THE OUTSIDE WORLD





Client-server interaction



Client

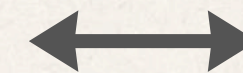
Communication

Server

- The browser accesses the webcam using standard Web APIs
- Video is not streamed continuously, single frames are captured
- Each captured frame is encoded as an image and sent to the backend via HTTP POST requests



- Communication is based on simple REST calls
- The client sends images to:
 - /identify* for recognition
 - /enroll* for user registration
- No biometric processing is performed on the client side



- Flask receives the image and runs the face recognition pipeline (detection, alignment and embedding extraction)
- A minimal JSON response is returned telling:
 1. if the person is recognized
 2. the associated identity

Recognition and enrollment logic

Identification

- A facial embedding is extracted from the input image and compared against all stored embeddings in the local database
- Similarity is computed using a distance-based metric; the identity is accepted only if the score exceeds a predefined decision threshold
- If no stored embedding satisfies the threshold, the subject is classified as Unknown

Face → Embedding → Compare → Threshold → Identity / Unknown

Enrollment

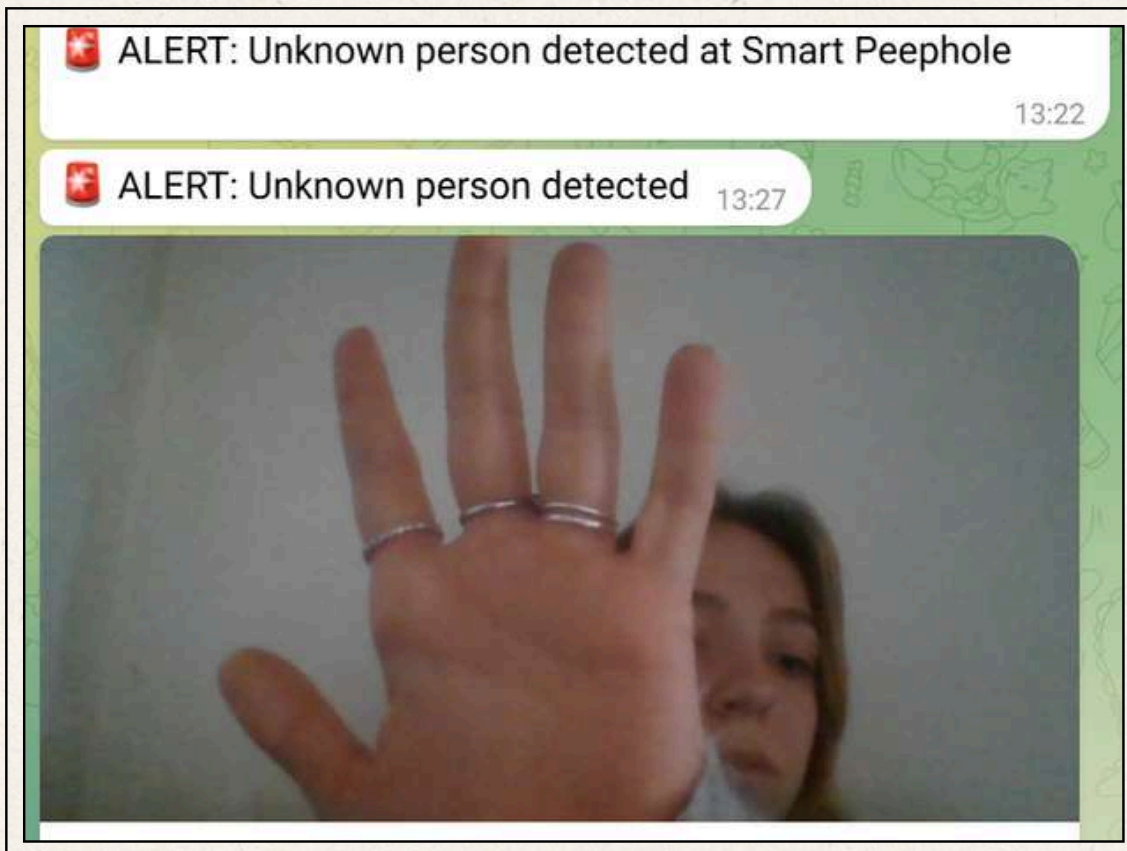
- Multiple images of the same user are required to improve robustness against pose, illumination and facial expression variations
- Extracted embeddings are stored and reused across sessions, enabling incremental growth of the dataset without retraining the model



Real-world interaction: unknown user alert

Immediate awareness for residents

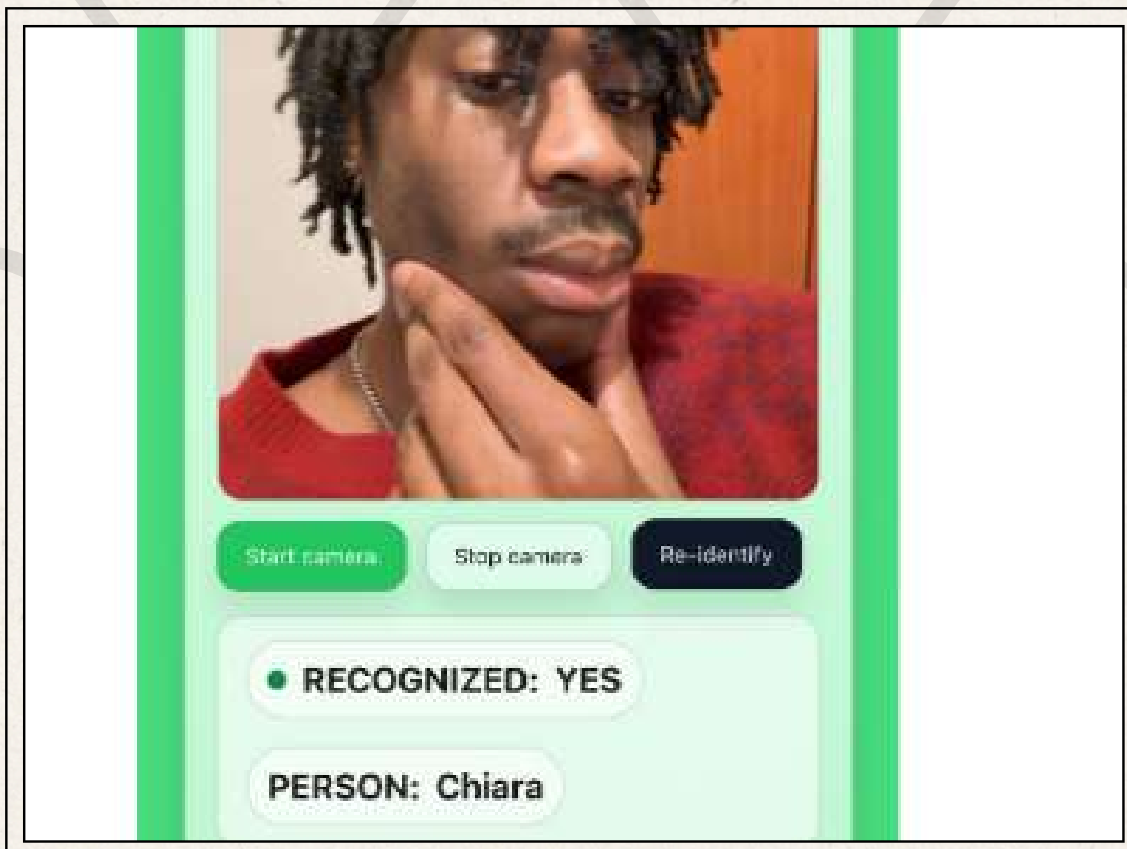
- 01** System detects unknown faces
- 02** Snapshot captured at detection time
- 03** Automatic notification sent to a Telegram group accessible via link





User feedback:

- 01** Ease of Use & Intuitiveness:
 - Users reported that the system is highly intuitive and easy to navigate, specifically the "one-click" camera activation and the enrollment process
 - The real-time interaction via Telegram Bot was highlighted as a very practical feature for domestic use
- 02** Early testing revealed some "identification hallucinations", where the system confused known users or failed to distinguish enrolled users
- 03** The most significant user complaint was that the system is host-dependent; it only functions when the local PC/Server is powered on and the ngrok tunnel is active



Strenghts and limitations - web side



- Simple and fast deployment, the application runs locally
- No cloud infrastructure required
- CPU-friendly performance

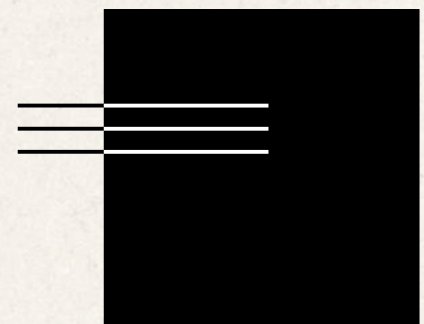


- Host-dependent availability
- Limited dataset size
- Ngrok URLs may change on the free plan
- Not designed for high-frame-rate surveillance



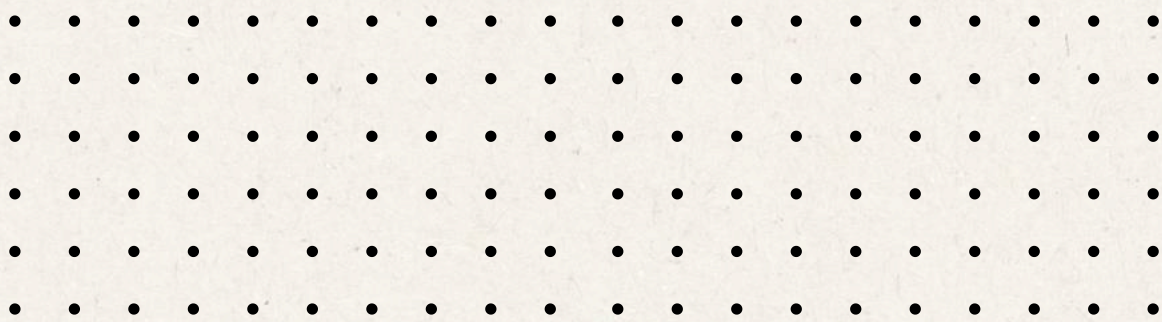
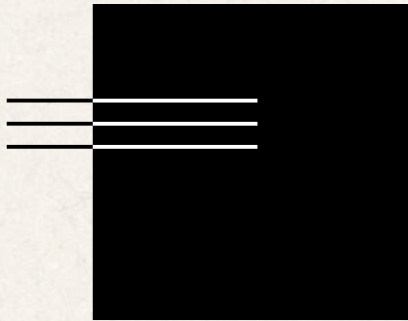
Conclusions and future directions

- **Effective pipeline implementation:** end-to-end face recognition system for domestic use, integrating Dlib for detection/alignment and SphereFace for high-dimensional feature extractio
- **Systematic performance benchmarking:** the project analyzes deployment constraints, computational trade-offs and practical usability on consumer devices
- **Hardware versatility & edge readiness:** the system demonstrates that reliable face recognition can be achieved on CPU-only hardware, making it suitable for low-cost and edge scenario
- **Privacy-oriented architecture:** By keeping the face database local and securing enrollment with an API Key, the system addresses core ethical and privacy concerns regarding biometric data management



Future directions

- **Enhanced ethical transparency:** developing a dedicated UI for user consent management, ensuring the system is fully compliant with evolving GDPR regulations for smart home technologies
- **Multi-Face identification support:** allow the peephole to simultaneously recognize all members of a group at the door by processing multiple detection regions in parallel
- **Integration of Image Quality Assessment:** implementing a pre-filtering module to evaluate the quality of captured frames to automatically discard blurred or low-light images
- **Edge and IoT Deployment:** testing on smartphones and IoT devices to find the best balance between speed and battery life for 24/7 usage



Thank you for your attention

Any questions?